

Microprocessor Systems

Lab5 - Universal Asynchronous Receiver Transmitter (UART)

Prof. Ren-Shuo Liu

NTHU EE



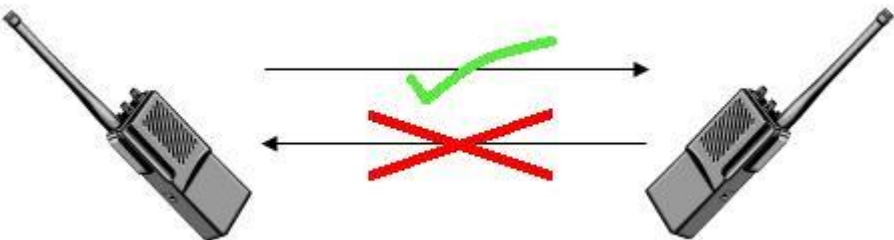
Introduction

- Universal asynchronous receiver-transmitter (UART) is an asynchronous serial communication.
- The data format and transmission speeds are configurable.
- Asynchronous communication doesn't require the clock to be connected between nodes.
- An UART has one connection for transmitted data, called TX, and another for received data, called RX.

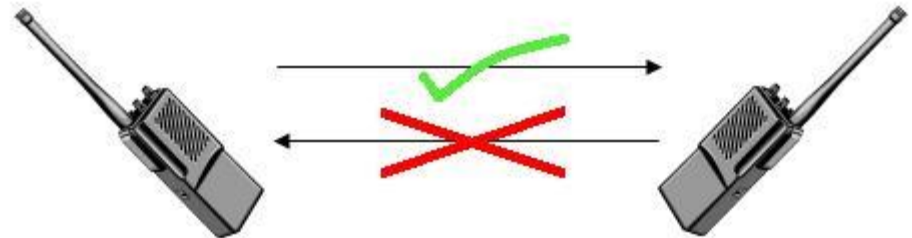
Communication

- Communication may be *simplex* , *full duplex* or *half duplex* .

Simplex

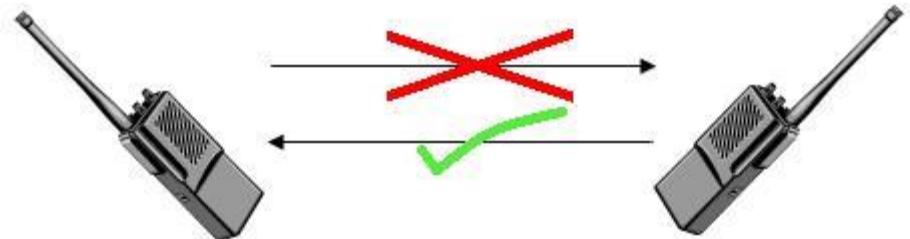


Half duplex



OR

Full duplex

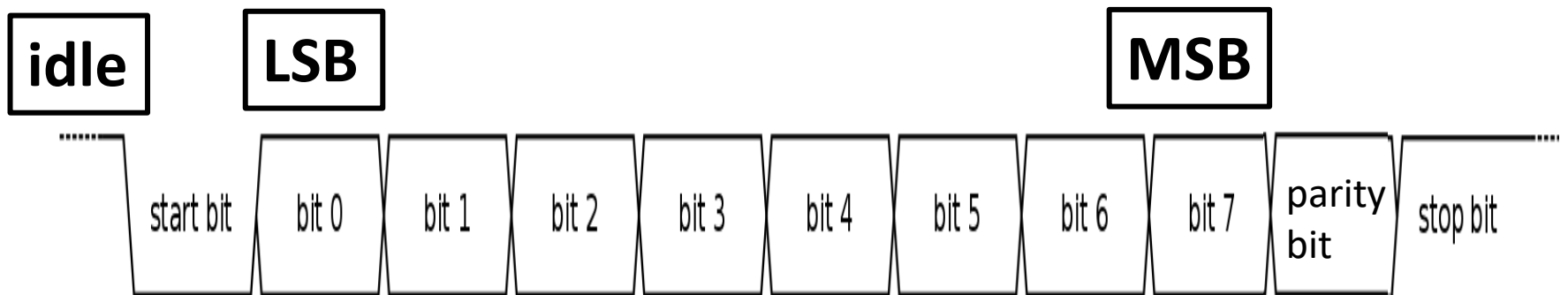


Common Configurations

- Data rate (or baud rate) are predetermined.
 - Both transmitter and receiver are pre-set to the same data rate.
- Standard baud rates include 110, 300, 600, 1200, 2400, 4800, 9600, 14400, 19200, 38400, 57600, 115200, 128000 and 256000 bits per second (bps).
- Data length are also predetermined to tell how many bits would transfer/receive (5 to 9 bits, typically).

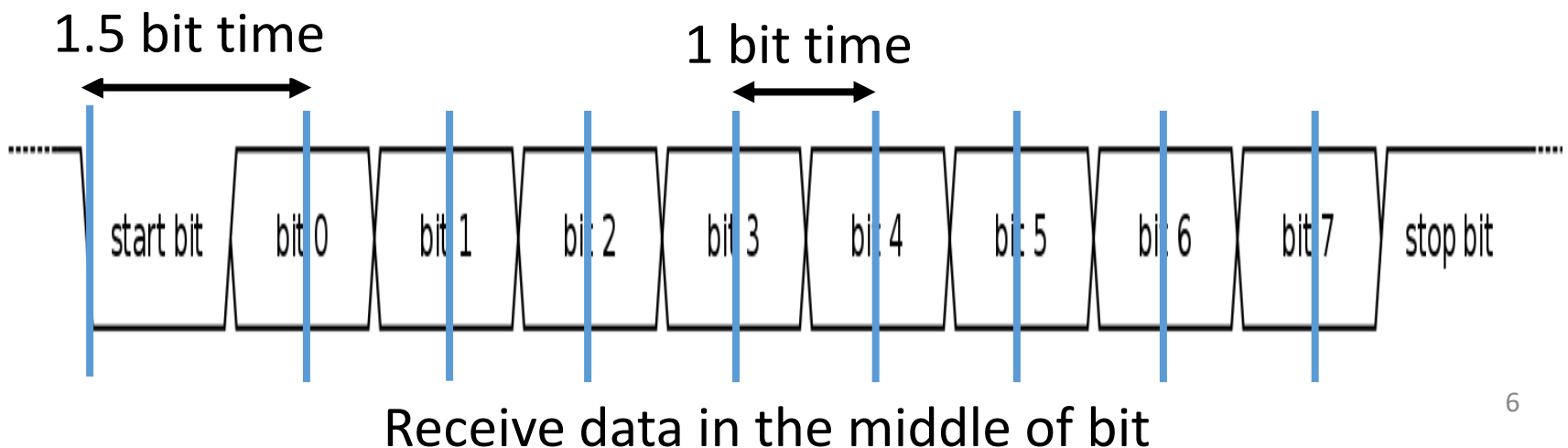
Data Frame

- Idle: keep logic high.
- The start bit (logic low) signals receiver that a data character is coming.
- If parity bit is used, it would be placed after the data bits to keep the package odd or even logic high.
- The final bit is the stop bit (logic high).



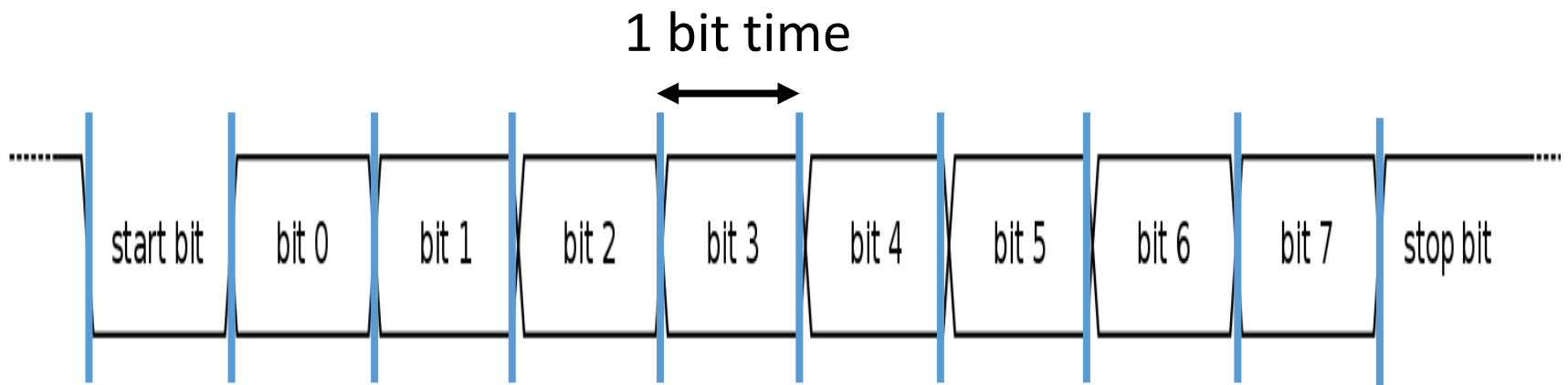
Receiver

- After the receiver getting the start bit, it starts to sample bit0 after 1.5 bit time in order to ensure that the data is stable, and then sample succeeding data every 1 bit time.
- After the required number of bit periods for the character length (5 to 9 bits) have elapsed, the contents of the shift register are made available.



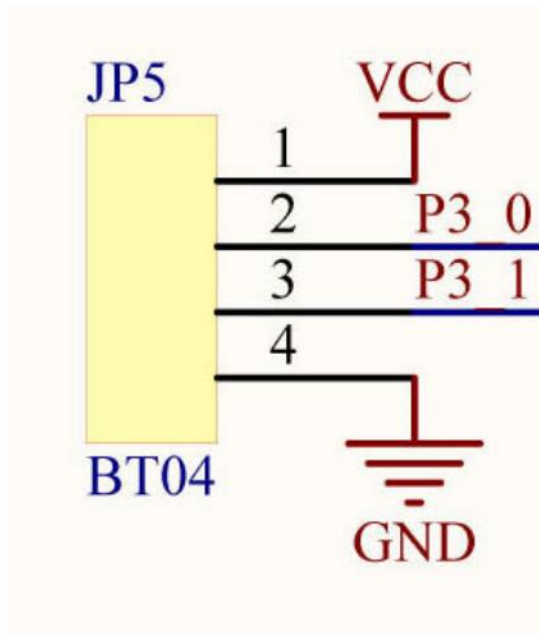
Transmitter

- As soon as the sending system deposits a character in the shift register, the UART generates a start bit.
- Shifts the required number of data bits out to the line, generates and sends the parity bit (if used), and sends the stop bits.

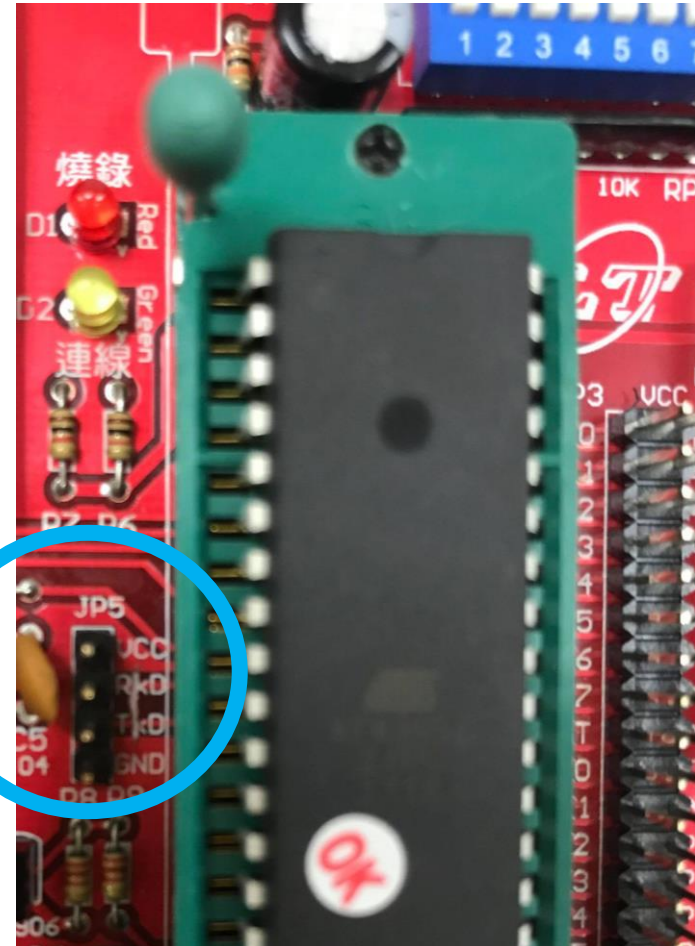


All transmit data is the same bit time

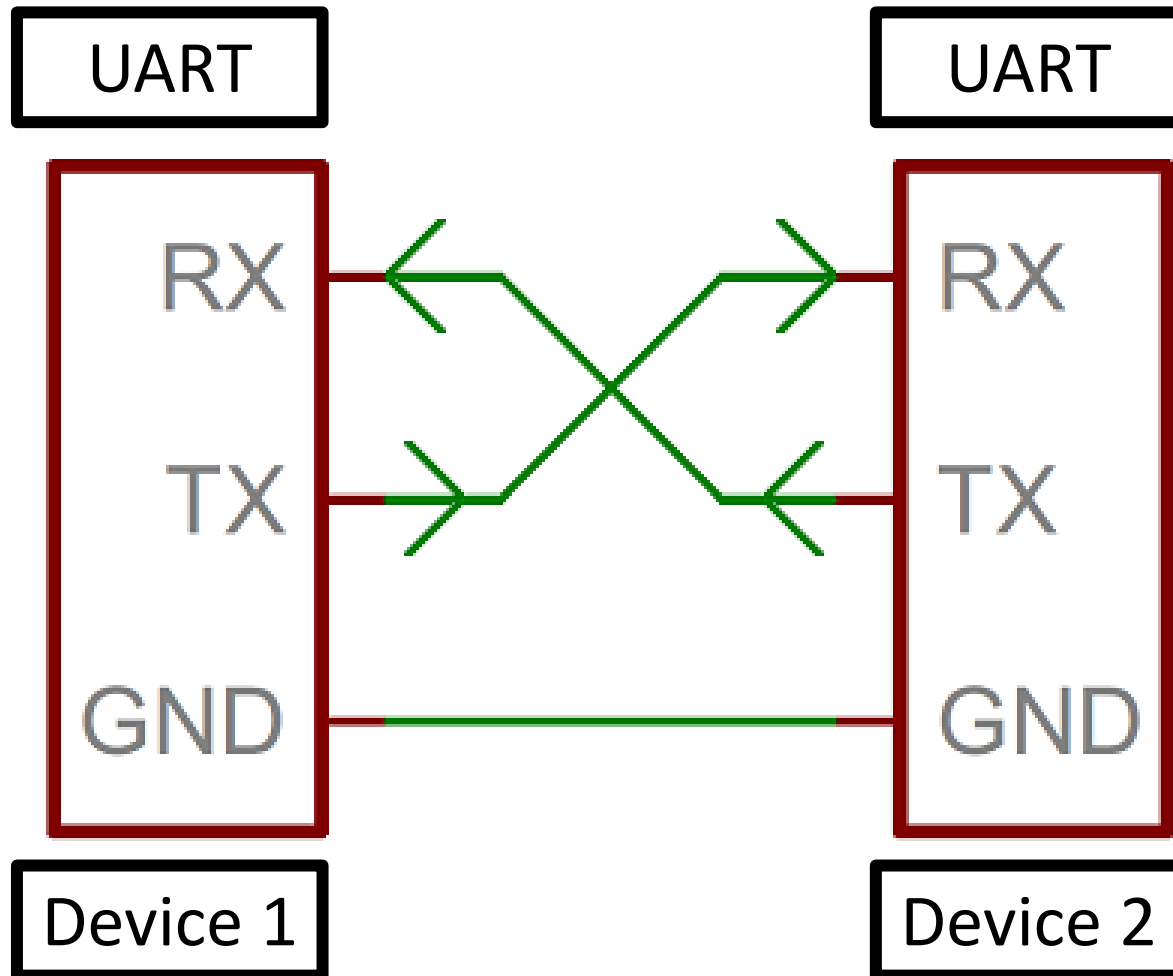
8051-UART Position



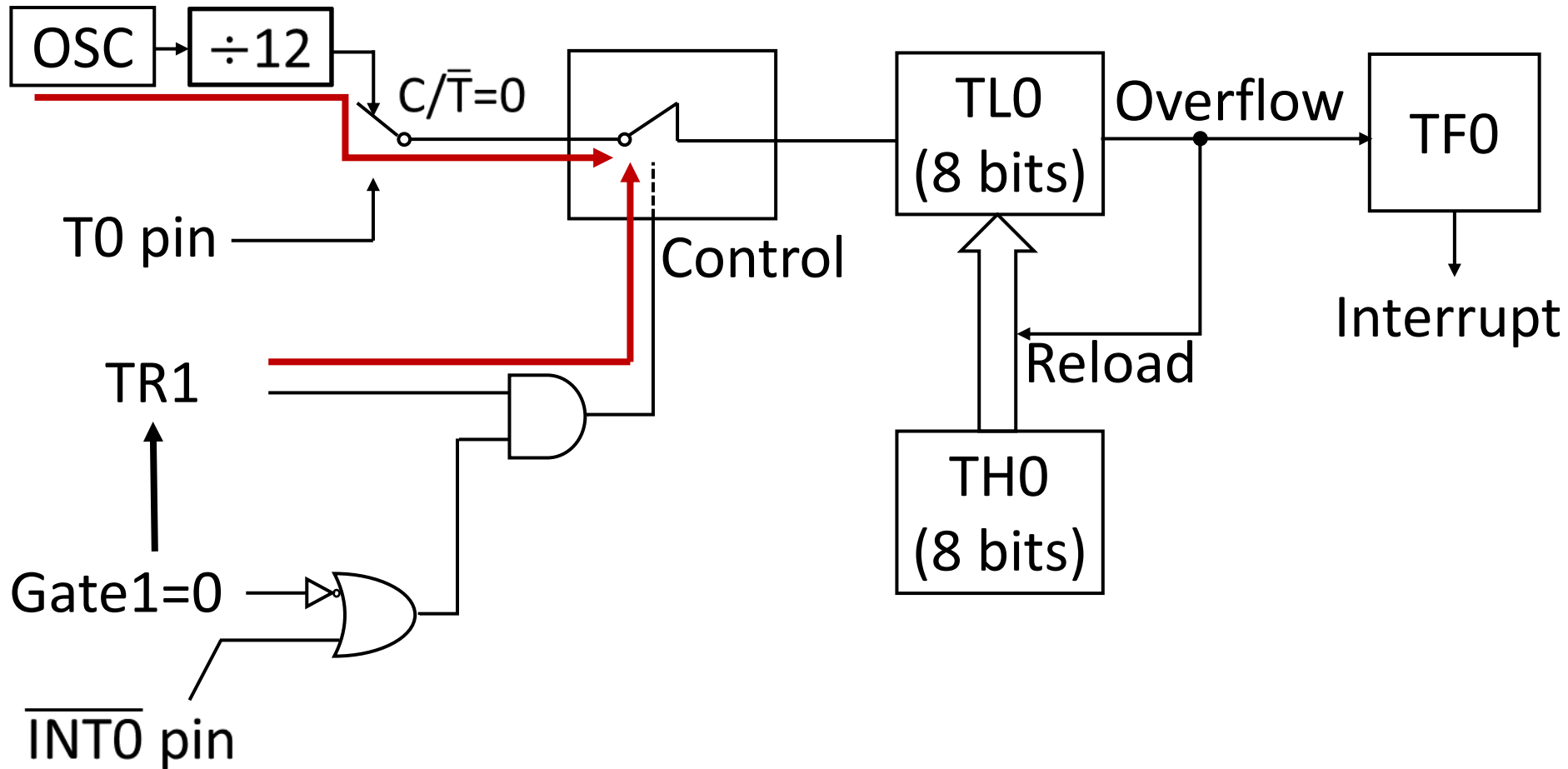
- P3_0: Rx
- P3_1: Tx



UART Communication

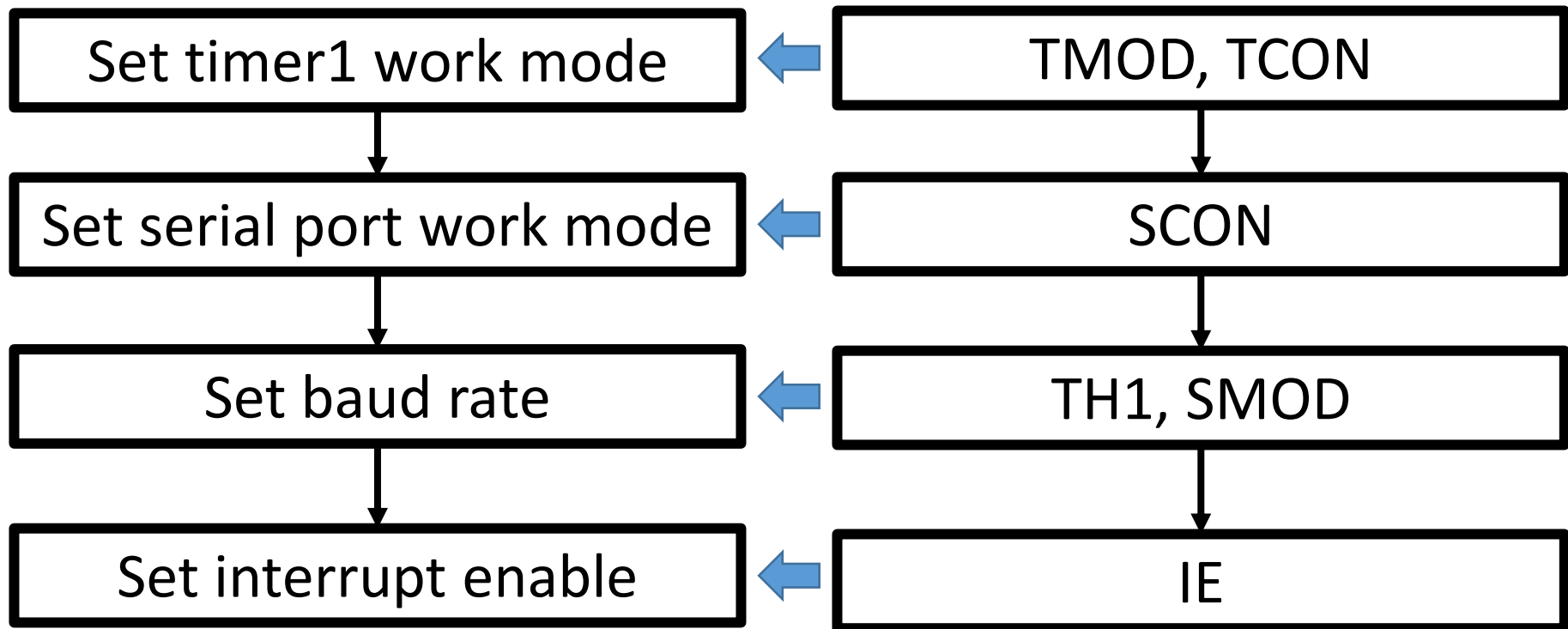


Timer1 Mode2 (8bit Operation with Auto Reload)



8051 Serial Port Settings

- Set special function registers in 8051



Time Mode Control Register (TMOD)

	Timer1				Timer0			
Bit No.	7	6	5	4	3	2	1	0
Symbol	Gate	C/ $\bar{T}$	M1	M0	Gate	C/ $\bar{T}$	M1	M0

M1	M0	Mode	Description
0	0	0	13-bit counter
0	1	1	16-bit counter
1	0	2	8-bit counter with autoreload
1	1	3	Split Timer0 into 8-bit counters or stop Timer1

- Gate1 (bit7): 1 for interrupt to drive timer.
0 for TR1 in TCON to drive Timer1.
- C/T: 1 for timer working as counter.
0 for timer working as timer.

Timer Control Register (TCON)

TMOD								
	Timer1				Timer0			
Bit No.	7	6	5	4	3	2	1	0
Symbol	Gate	$C/\bar{T}$	M1	M0	Gate	$C/\bar{T}$	M1	M0

TCON								
Bit No.	7	6	5	4	3	2	1	0
Symbol	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

- Set Gate1=0 (TMOD bit7) and TR1=1 to open timer 1

Serial Port Control Register (SCON)

Bit No.	7	6	5	4	3	2	1	0
Symbol	SM0	SM1	SM2	REN	TB8	RB8	TI	RI

SM0	SM1	Mode	Description
0	0	0	Shift register
0	1	1	8 Bit UART with configurable baud rate
1	0	2	9 Bit UART with identical baud rate
1	1	3	9 Bit UART with configurable baud rate

- Set to mode 1.
- Ren: set/cleared to enable/disable reception.
- TI: Transmitter interrupt flag.
- RI: Receiver interrupt flag.

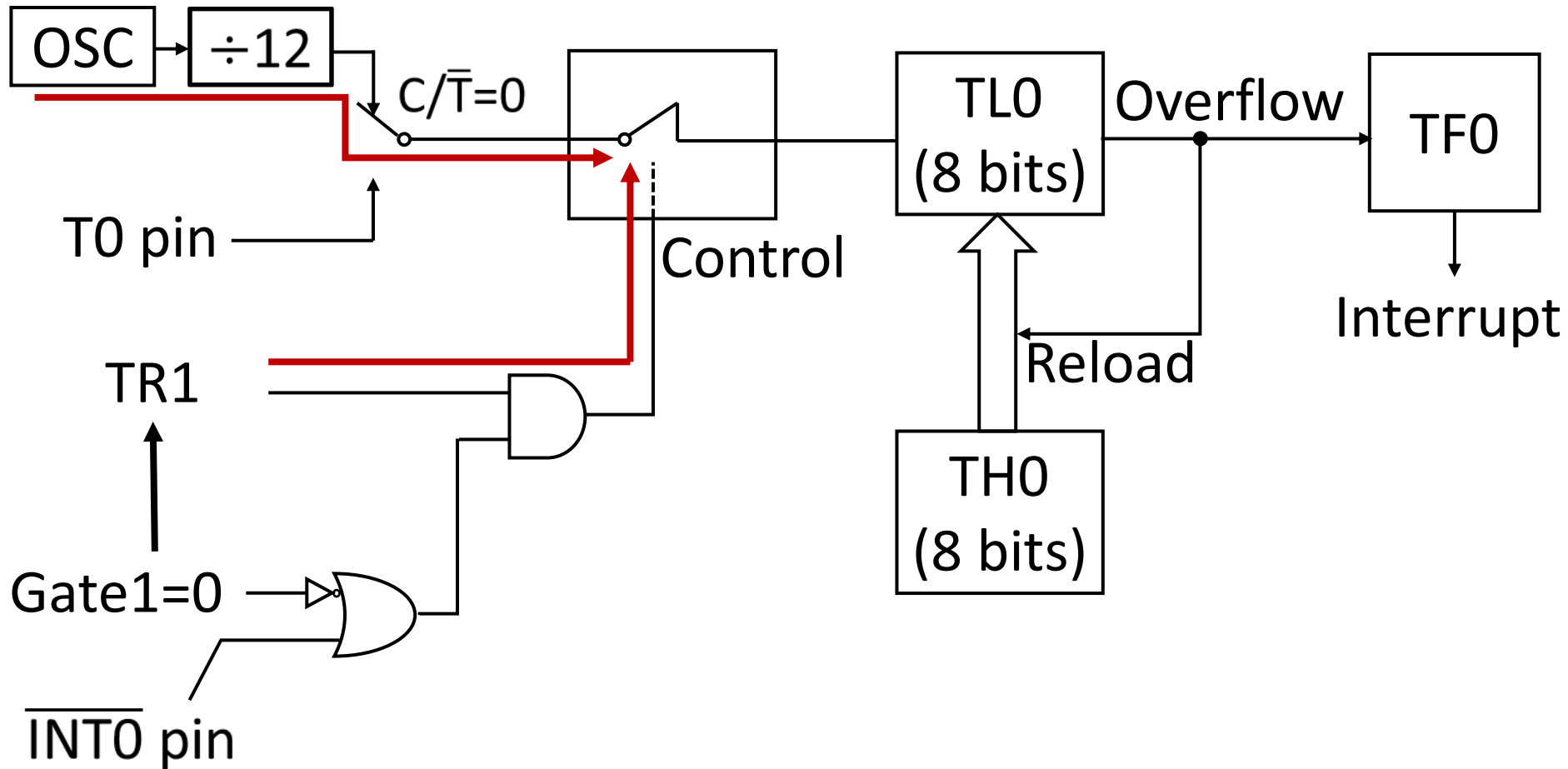
IE (Interrupt Enable) Register

Bit No.	7	6	5	4	3	2	1	0
Symbol	EA	RSV	RSV	ES	ET1	EX1	ET0	EX0

EA	Enables interrupt
-	Reserved for future use
-	Reserved for future use
ES	Enables/disables serial port interrupt
ET1	Enables/disables timer1 overflow interrupt
EX1	Enables/disables external interrupt1
ET0	Enables/disables timer0 overflow interrupt
EX0	Enables/disables external interrupt0

- Set EA and ES =1 (Enable serial port interrupt)

Timer1 Mode2 (8bit Operation with Auto Reload)



UART Clock Source (Case 1)

- $\text{Freq_T1} = \text{Freq_osc} / (12 * (256 - \text{TH1}))$ (C/T = 0)
- $\text{Freq_Uart} = \text{Baudrate} = \text{Freq_T1} / 32$ (SMOD = 0)
 $\text{Freq_Uart} = \text{Baudrate} = \text{Freq_T1} / 16$ (SMOD = 1)
- For Example:
 $\text{Freq_osc} = 12\text{Mhz}$, $\text{Baudrate} = 9600\text{bps}$, $\text{SMOD} = 1$
- $\text{Freq_T1} = 9600 * 16 = 153,600$
 $(256 - \text{TH1}) = 12\text{M} / 12 / 153.6\text{k} = 6.51$
 $\text{TH1} = 249 \text{ or } 250$
The actual Baudrate = 8928 or 10416 bps
error = -7.5% or +7.8% (> +- 6%) can't use

$$\text{TH1} = 256 - \frac{2^{\text{SMOD}} \times \text{oscillator frequency}}{32 \times 12 \times \text{baudrate}}$$

UART Clock Source (Case 2)

- $\text{Freq_T1} = \text{Freq_osc} / (12 * (256 - \text{TH1}))$ (C/T = 0)
- $\text{Freq_Uart} = \text{Baudrate} = \text{Freq_T1} / 32$ (SMOD = 0)
 $\text{Freq_Uart} = \text{Baudrate} = \text{Freq_T1} / 16$ (SMOD = 1)
- For Example:
 $\text{Freq_osc} = 11.0592\text{Mhz}$, $\text{Baudrate} = 9600\text{bps}$, $\text{SMOD} = 1$
- $\text{Freq_T1} = 9600 * 16 = 153,600$
 $(256 - \text{TH1}) = 11.0592\text{M} / 12 / 153.6\text{k} = 6$
 $\text{TH1} = 250$
The actual Baudrate = 9600 bps
error = 0%

$$\text{TH1} = 256 - \frac{2^{\text{SMOD}} \times \text{oscillator frequency}}{32 \times 12 \times \text{baudrate}}$$

UART Clock Source (Case 3)

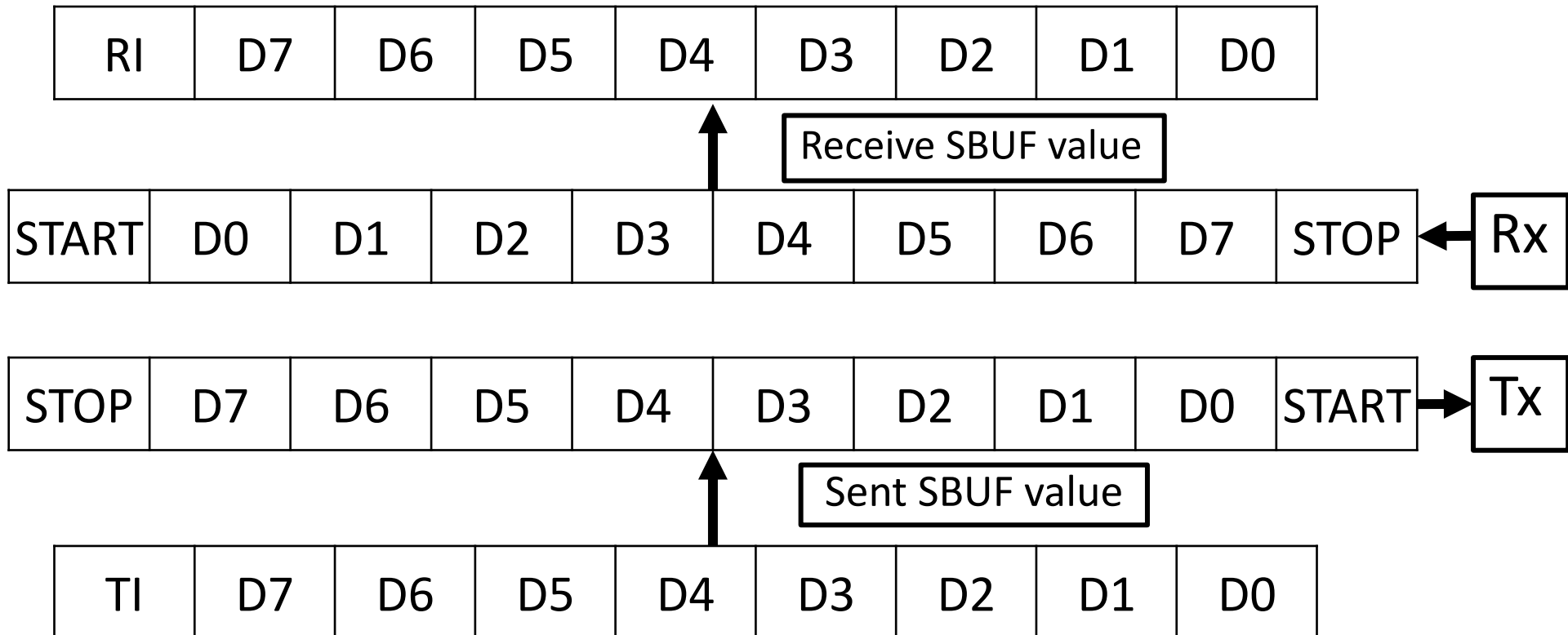
User define baudrate

- $\text{Freq_T1} = \text{Freq_osc} / (12 * (256 - \text{TH1}))$ (C/T = 0)
- $\text{Freq_Uart} = \text{Baudrate} = \text{Freq_T1} / 32$ (SMOD = 0)
 $\text{Freq_Uart} = \text{Baudrate} = \text{Freq_T1} / 16$ (SMOD = 1)
- For Example:
 $\text{Freq_osc} = 12\text{Mhz}$, $\text{Baudrate} = 8928\text{bps}$, $\text{SMOD} = 1$
- $\text{Freq_T1} = 8928 * 16 = 142,848$
 $(256 - \text{TH1}) = 12\text{M} / 12 / 142.848\text{k} = 7$
 $\text{TH1} = 249$
The actual Baudrate = 8928
error = 0%

$$\text{TH1} = 256 - \frac{2^{\text{SMOD}} \times \text{oscillator frequency}}{32 \times 12 \times \text{baudrate}}$$

Serial Buffer SBUF

- After setting, the data would transmit/receive through SBUF.



Example code : Initial Setting

```
// set timer and initialize communication port
TMOD = 0x20;           // set timer1 to mode2
SCON = 0x50;           // set serial mode1
TH1 = 250;             // baudrate = 9600, fosc = 11.0592MHz
TR1 = 1;               // enable timer = 1
IE = 0x90;             // enable serial port interrupt
```

Example code : Tx sends data

```
// sending
if (send_flag==1)
{
    SBUF = send_data;    // send data to SBUF
    send_flag = 0;
}
```

Example code : Rx receives data

```
// after receiving the data, show the result on 7-segment display
if(received_flag == 1) {
    Write7219(1, received_data);
    received_flag = 0;
}
```

Example Code : Interrupt

Transmitter

```
void uart_isr(void) __interrupt (4)
{
    if (TI == 1)
        TI = 0;
}
```

Receiver

```
void uart_isr(void) __interrupt (4)
{
    if( RI == 1) {
        RI = 0;                // refresh RI
        received_data = SBUF;   // receive data from SBUF
        received_flag = 1;      // receiving finished
    }
}
```